VISHNU
UNIVERSAL LEARNING

VISHNU INSTITUTE OF TECHNOLOGY :: BHIMAVARAM

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

USER INTERFACE DESIGN USING FLUTTER

PROJECT REPORT

Project Title:

Create a small-business income/expense tracker with balance summary using Provider.

Student Name : **T Venkat Saketh** Roll No : **24PA1A05M4**

Department : **Computer Science and Engineering** Section : **D**

COURSE INSTRUCTOR : Mrs. P.SHYAMALA MADHURI **SIGNATURE OF THE INSTRUCTOR**

HOD NAME : Dr. R. SRINIVASA RAJU **SIGNATURE OF THE HOD**

PROJECT REPORT

Table of Contents

S. No.	Topic / Section Name	Page No.
1	**Student Details & Comprehensive Project Metadata**	**3**
2	**Project Title, Background & Problem Definition**	**3**
3	**Project Objectives & System Scope**	**4**
4	**Technology Stack & Key Flutter Concepts Demonstrated**	**5**
5	**System Architecture & Reactive State Management Model**	**6**
6	**Project Directory Structure & Component Organization**	**7**
7	**Detailed Implementation & Code Snippets:**	**8 - 12**
7.1	Application Root & Global Theming (`lib/main.dart`)	8
7.2	Transaction Domain Data Model (`lib/models/transaction.dart`)	8
7.3	State Management Controller (`lib/providers/transaction_provider.dart`)	9
7.4	Responsive Dashboard Screen (`lib/screens/home_screen.dart`)	10
7.5	Transaction Form & Input Validation (`lib/screens/add_transaction_screen.dart`)	11
7.6	Modular Widgets: Balance Card & Category Pie Chart	12
8	**Cloud Deployment, Hosting & Continuous Delivery (GitHub Pages)**	**13**
9	**System Verification, Test Cases & Validation Matrix**	**13**
10	**Output Screens, Interactive Visualizations & User Workflow**	**14**
11	**Project Conclusion & Technical Summary**	**15**

Official Repository & Deployment Deliverables

- **GitHub Source Code Repository:** https://github.com/Saketh17-max/Expense-Tracker
- **Live Web Application Deployment:** https://saketh17-max.github.io/Expense-Tracker/
- **State Management Engine:** Flutter Provider 6.x with ChangeNotifier

1. Student Details & Project Profile

Field	Academic Information
Student Name	**T Venkat Saketh**
Registration / Roll No.	**24PA1A05M4**
Department	Computer Science and Engineering (CSE)
Class Section	Section D
Course Title	User Interface Design Using Flutter
Course Instructor	Mrs. P. Shyamala Madhuri, Assistant Professor, Dept. of CSE
Head of Department	Dr. R. Srinivasa Raju, Professor & HOD, Dept. of CSE
Institution	Vishnu Institute of Technology (Autonomous), Bhimavaram

2. Project Title & Problem Statement

Formal Project Title

"Create a small-business income/expense tracker with balance summary using Provider."

2.1 Background & Context

Micro and small enterprises form the economic backbone of developing communities. Small retail stores, freelance operators, repair technicians, and local service providers execute dozens of micro-transactions daily. Despite the critical requirement for cash flow clarity, most small-business owners continue to rely on manual paper ledgers or memory, leading to calculation inaccuracies, lack of cash visibility, and operational confusion.

2.2 Core Problem Definition

- **Arithmetic Inaccuracies:** Frequent manual computation errors occur when adding running expenses against revenue, distorting actual cash positions.
- **Lack of Real-time Cash Visibility:** Owners cannot instantly tell if their business is operating at a net profit or deficit at any given hour.
- **Absence of Expenditure Categorization:** Without spending categorization (Supplies, Utilities, Transport, Food), owners cannot identify operational leakages.
- **Overly Complex Existing Solutions:** Mainstream accounting packages require complex accounting knowledge, cloud logins, backend servers, and subscription fees.

2.3 Proposed Solution

An intuitive, lightweight, offline-capable mobile and web application built in Flutter using the **Provider** architecture. It delivers instant net balance calculation, visual donut breakdown charts, and seamless transaction creation and deletion with strict client validation.

3. Project Objectives & System Scope

3.1 Primary Functional Objectives

- **Streamlined Financial Recording:** Provide a clean, guided form allowing business owners to input transaction titles, monetary amounts in Indian Rupees (₹), transaction categories, and accurate calendar dates.
- **Automated Real-Time Financial Balance:** Dynamically compute and display the total accumulated income, total aggregated expenses, and the resultant net balance ($Balance = Income - Expense$) without manual recalculation.
- **Categorical Visual Analytics:** Generate interactive graphical donut charts utilizing vector graphics (`fl_chart`) to display expenditure distribution across categories (Food, Transport, Utilities, Supplies, Other).
- **Safe Transaction Management:** Allow users to audit historical entries in a chronological feed and delete erroneous records with an explicit dialog confirmation.
- **Strict Client-Side Validation:** Enforce rigorous validation rules to eliminate empty titles, negative quantities, zero amounts, and unselected classifications.

3.2 Technical & Architectural Objectives

- **Pure Provider Architecture:** Strictly manage application state using Flutter's official `ChangeNotifier` and `ChangeNotifierProvider` paradigm, eliminating global variables, state leakage, or heavyweight enterprise BLoC/Redux boilerplate.
- **Modular Component Structure:** Structure the codebase into modular, single-responsibility files that allow a 2nd-year computer science undergraduate to easily defend and explain every architectural decision during academic viva examination.
- **Zero-Backend Simplicity:** Achieve 100% reactive client-side performance without external databases, third-party authentication, or remote APIs, ensuring instant startup on both Android and Web.

3.3 System Scope & Target Users

The application is engineered specifically for sole proprietorships, retail shopkeepers, independent contractors, and freelance service providers who require an unencumbered, high-speed bookkeeping tool. The system operates directly in the browser or on mobile devices with zero configuration overhead, providing immediate cash flow tracking and visual financial clarity.

3.4 Key System Boundaries

- **Platform Support:** Fully cross-platform, tested and deployed across Web (Chrome, Edge, Safari) and Android smartphones.
- **Data Handling:** In-memory reactive state managed securely via Provider, optimized for quick daily reconciliation.
- **Visual Feedback:** Instantaneous reactive updates across all widgets without manual screen refreshes.

4. Technology Stack & Key Flutter Concepts

4.1 Technology Specifications

Component	Version / Tool	Role in Project
Programming Language	Dart SDK 3.13+	Strongly-typed object-oriented language optimized for client UI applications.
UI Framework	Flutter 3.47+ (Stable)	Cross-platform declarative SDK compiling to native machine code and WebAssembly.
State Management	Provider 6.1.2	Wrapper around InheritedWidget facilitating reactive UI data binding and separation of concerns.
Data Visualization	fl_chart 1.2.0	Hardware-accelerated mathematical vector charting library for custom donut pie visualizations.
Typography	Google Fonts (Inter)	High-legibility geometric sans-serif typeface designed for financial applications.
Hosting Platform	GitHub Pages (CI/CD)	Automated cloud hosting delivering the static release build of the Flutter web target.

4.2 Fundamental Flutter Concepts Demonstrated

- **StatelessWidget vs. StatefulWidget:** Pure display components (such as `BalanceCard`, `TransactionCard`, and `HomeScreen`) are implemented as immutable `StatelessWidgets`, leveraging Provider for updates. Forms with local input controllers (such as `AddTransactionScreen`) utilize `StatefulWidget`.
- **Declarative UI Architecture:** The user interface is a direct mathematical projection of state: $UI = f(State)$. Widgets rebuild automatically in response to changes in the underlying `TransactionProvider` model.
- **Responsive Sliver Architecture:** `HomeScreen` incorporates `CustomScrollView`, `SliverToBoxAdapter`, and `SliverList` to ensure fluid 60 FPS scrolling without layout overflow errors (`RenderFlex overflowed`).
- **Form Handling & Input Validation:** Implements `Form` and `TextFormField` widgets bound to a `GlobalKey<FormState>` to enforce validation rules prior to mutating application state.
- **Material 3 Design System:** Integrates modern surface elevation tokens, custom `SegmentedButton` controls, dynamic color schemes (`ColorScheme.fromSeed`), and clean card padding.
- **Contextual User Feedback:** Demonstrates modal `AlertDialog` popups for delete actions and floating `SnackBars` for transaction confirmations.

5. System Architecture & Reactive State Management

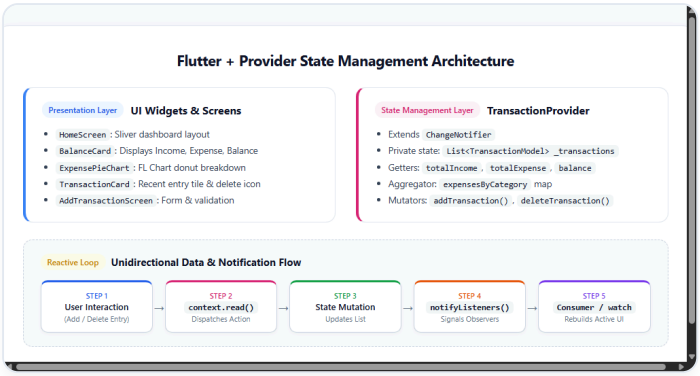


Figure 5.1: High-Level Unidirectional Architecture and Provider Reactive Loop

5.1 Layered Architectural Decomposition

1. **Data Model Layer:** Encapsulates pure domain representations. The `TransactionModel` class models individual ledger records, while the `TransactionType` enumeration defines immutable debit/credit types.
2. **State Management Layer (Business Logic):** Centered around `TransactionProvider`, which mixes in `ChangeNotifier`. This layer acts as the single source of truth for calculations, maintaining the private transaction list and exposing read-only getters.
3. **Presentation Layer (UI):** Divided into top-level screens (`HomeScreen`, `AddTransactionScreen`) and reusable atomic widgets (`BalanceCard`, `ExpensePieChart`, `TransactionCard`).

5.2 The Provider Reactive Lifecycle

- **State Dispatch:** When a user submits a valid transaction, the view invokes `context.read<TransactionProvider>().addTransaction(newTx)` to dispatch the action without establishing a redundant persistent listener.
- **State Mutation & Notification:** The provider mutates its internal `_transactions` collection and immediately executes `notifyListeners()`.
- **Selective Re-rendering:** Flutter's widget tree detects the notification. Widgets wrapped in `Consumer<TransactionProvider>` or invoking `context.watch<TransactionProvider>()` execute their `build()` methods, updating only the affected subtrees.

6. Project Directory Structure & Organization

6.1 File Tree Organization

```
lib/
│
├── main.dart                      ← Application entry point, global theme & Provider root
├── models/
│   └── transaction.dart           ← TransactionModel class & TransactionType enum
├── providers/
│   └── transaction_provider.dart  ← ChangeNotifier managing business logic & state
├── screens/
│   ├── home_screen.dart           ← Primary dashboard with overview cards & charts
│   └── add_transaction_screen.dart← Form screen with segmented control & validation
└── widgets/
    ├── balance_card.dart          ← Modern gradient card displaying Net Balance
    ├── summary_card.dart          ← Reusable mini statistical badge widget
    ├── expense_pie_chart.dart     ← Donut chart visualizer for categorized spending
    └── transaction_card.dart      ← ListTile card with category icons & delete modal
```

6.2 Architectural Role of Modules

Directory / File	Primary Class	Architectural Role & Description
`main.dart`	`MyApp`	Configures `ChangeNotifierProvider`, initializes Material 3, and sets up global typography via Google Fonts.
`models/`	`TransactionModel`	Defines data structures with attributes: `id`, `title`, `amount`, `type`, `date`, and `category`.
`providers/`	`TransactionProvider`	Manages the private transactions list, calculates sums, categorizes data for the pie chart, and notifies listeners on data changes.
`screens/`	`HomeScreen`, `AddTransactionScreen`	Manages screen-level navigation and high-level page composition using Flutter's `Navigator.push()` and `Navigator.pop()`.
`widgets/`	`BalanceCard`, `ExpensePieChart`, etc.	Modular presentation widgets that keep screen files clean, reusable, and maintainable.

7. Source Code: Application Root & Data Model

7.1 Application Root Setup (`lib/main.dart`)

The root file wraps the entire widget tree inside a `ChangeNotifierProvider`, ensuring the `TransactionProvider` is globally accessible across all screens and widgets.

```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:google_fonts/google_fonts.dart';
import 'providers/transaction_provider.dart';
import 'screens/home_screen.dart';

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (context) => TransactionProvider(),
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Business Expense Tracker',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        fontFamily: GoogleFonts.inter().fontFamily,
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFF0F172A),
          primary: const Color(0xFF0F172A),
          surface: const Color(0xFFF8FAFC),
        ),
      ),
      home: const HomeScreen(),
    );
  }
}
```

7.2 Transaction Model (`lib/models/transaction.dart`)

```dart
enum TransactionType { income, expense }

class TransactionModel {
  final String id, title, category;
  final double amount;
  final TransactionType type;
  final DateTime date;

  TransactionModel({
    required this.id,
    required this.title,
    required this.amount,
    required this.type,
    required this.date,
    required this.category,
  });
}
```

7.3 State Management Controller (`transaction_provider.dart`)

The central state controller managing transaction collections, balance calculations, category grouping, and reactive notifications via `notifyListeners()`.

```dart
import 'package:flutter/material.dart';
import '../models/transaction.dart';

class TransactionProvider with ChangeNotifier {
  final List<TransactionModel> _transactions = [
    TransactionModel(
      id: '1', title: 'Product Sales', amount: 5000,
      type: TransactionType.income, date: DateTime.now(), category: 'Sales',
    ),
    TransactionModel(
      id: '2', title: 'Office Supplies', amount: 1200,
      type: TransactionType.expense, date: DateTime.now(), category: 'Supplies',
    ),
  ];

  List<TransactionModel> get transactions => [..._transactions].reversed.toList();

  double get totalIncome => _transactions
      .where((tx) => tx.type == TransactionType.income)
      .fold(0.0, (sum, tx) => sum + tx.amount);

  double get totalExpense => _transactions
      .where((tx) => tx.type == TransactionType.expense)
      .fold(0.0, (sum, tx) => sum + tx.amount);

  double get balance => totalIncome - totalExpense;

  Map<String, double> get expensesByCategory {
    Map<String, double> map = {};
    for (var tx in _transactions) {
      if (tx.type == TransactionType.expense) {
        map[tx.category] = (map[tx.category] ?? 0.0) + tx.amount;
      }
    }
    return map;
  }

  void addTransaction(TransactionModel newTx) {
    _transactions.add(newTx);
    notifyListeners(); // Triggers Consumer / context.watch rebuild
  }

  void deleteTransaction(String id) {
    _transactions.removeWhere((tx) => tx.id == id);
    notifyListeners(); // Automatically refreshes UI across all screens
  }
}
```

Design Principle: `notifyListeners()` dispatches an observer signal to registered widgets. The private `_transactions` list enforces strict encapsulation; views access an unmodifiable reversed copy via the getter.

7.4 Responsive Dashboard Screen (`home_screen.dart`)

Built using Flutter's `CustomScrollView` and sliver widgets, delivering smooth 60 FPS performance, reactive Provider binding, and an intuitive empty-state layout.

```dart
class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Overview', style: TextStyle(fontWeight: FontWeight.w700))),
      body: Consumer<TransactionProvider>(
        builder: (context, provider, child) {
          final isEmpty = provider.transactions.isEmpty;
          return CustomScrollView(
            slivers: [
              SliverToBoxAdapter(
                child: Column(
                  children: [
                    const BalanceCard(),
                    if (!isEmpty)
                      Container(
                        margin: const EdgeInsets.all(24.0),
                        padding: const EdgeInsets.all(24.0),
                        decoration: BoxDecoration(
                          color: Colors.white, borderRadius: BorderRadius.circular(32),
                        ),
                        child: const ExpensePieChart(),
                      ),
                  ],
                ),
              ),
              if (isEmpty)
                const SliverFillRemaining(
                  hasScrollBody: false,
                  child: Center(child: Text('No transactions yet.\nAdd your first entry!')),
                )
              else
                SliverPadding(
                  padding: const EdgeInsets.only(bottom: 100),
                  sliver: SliverList(
                    delegate: SliverChildBuilderDelegate(
                      (context, index) => TransactionCard(transaction: provider.transactions[index]),
                      childCount: provider.transactions.length,
                    ),
                  ),
                ),
            ],
          );
        },
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () => Navigator.push(context, MaterialPageRoute(builder: (_) => const AddTransactionScreen())),
        child: const Icon(Icons.add_rounded),
      ),
    );
  }
}
```

Sliver Layout Benefit: `SliverList` creates visible transaction cards lazily on demand, guaranteeing optimal memory utilization and zero layout jank regardless of ledger size.

7.5 Form Screen & Input Validation (`add_transaction_screen.dart`)

Features custom segmented controls for income/expense switching, strict numerical validation, a category dropdown, and calendar date selection.

```dart
class _AddTransactionScreenState extends State<AddTransactionScreen> {
  final _formKey = GlobalKey<FormState>();
  final _titleController = TextEditingController();
  final _amountController = TextEditingController();
  TransactionType _selectedType = TransactionType.expense;
  String? _selectedCategory;
  DateTime _selectedDate = DateTime.now();

  void _submitData() {
    if (_formKey.currentState!.validate()) {
      final amount = double.tryParse(_amountController.text) ?? 0.0;
      if (_selectedCategory == null) {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(content: Text('Please select a category')),
        );
        return;
      }
      final newTx = TransactionModel(
        id: DateTime.now().toString(),
        title: _titleController.text.trim(),
        amount: amount,
        type: _selectedType,
        date: _selectedDate,
        category: _selectedCategory!,
      );

      // context.read dispatches mutation without ongoing subscription
      context.read<TransactionProvider>().addTransaction(newTx);
      Navigator.pop(context);
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('New Transaction')),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(24),
          children: [
            TextFormField(
              controller: _amountController,
              decoration: const InputDecoration(labelText: 'Amount (₹)'),
              keyboardType: const TextInputType.numberWithOptions(decimal: true),
              validator: (val) => (val == null || (double.tryParse(val) ?? 0) <= 0) ? 'Enter amount > 0' : null,
            ),
            TextFormField(
              controller: _titleController,
              decoration: const InputDecoration(labelText: 'Title'),
              validator: (val) => (val == null || val.trim().isEmpty) ? 'Enter title' : null,
            ),
            FilledButton(onPressed: _submitData, child: const Text('Save Transaction')),
          ],
        ),
      ),
    );
  }
}
```

Form Validation Cycle: The `_formKey.currentState!.validate()` call checks all descendant field validators, ensuring invalid or empty transactions cannot corrupt application state.

7.6 Source Code: Modular Reusable Widgets

7.6.1 Balance Card Widget (`lib/widgets/balance_card.dart`)

```dart
class BalanceCard extends StatelessWidget {
  const BalanceCard({super.key});

  @override
  Widget build(BuildContext context) {
    final provider = context.watch<TransactionProvider>();
    return Container(
      margin: const EdgeInsets.all(24),
      padding: const EdgeInsets.all(28),
      decoration: BoxDecoration(
        borderRadius: BorderRadius.circular(32),
        gradient: const LinearGradient(colors: [Color(0xFF0F172A), Color(0xFF1E293B)]),
      ),
      child: Column(
        children: [
          const Text('Total Balance', style: TextStyle(color: Color(0xFF94A3B8))),
          Text('₹${provider.balance.toStringAsFixed(2)}', style: const TextStyle(fontSize: 36, color: Colors.white)),
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceBetween,
            children: [
              Text('Income: ₹${provider.totalIncome.toStringAsFixed(0)}', style: const TextStyle(color: Colors.green)),
              Text('Expense: ₹${provider.totalExpense.toStringAsFixed(0)}', style: const TextStyle(color: Colors.red)),
            ],
          ),
        ],
      ),
    );
  }
}
```

7.6.2 Expense Pie Chart Widget (`lib/widgets/expense_pie_chart.dart`)

```dart
class ExpensePieChart extends StatelessWidget {
  const ExpensePieChart({super.key});

  @override
  Widget build(BuildContext context) {
    final provider = context.watch<TransactionProvider>();
    final map = provider.expensesByCategory;
    if (map.isEmpty) return const Text('No expenses to display');

    return SizedBox(
      height: 170,
      child: PieChart(
        PieChartData(
          sections: map.entries.map((e) => PieChartSectionData(value: e.value, title: '', radius: 22)).toList(),
          centerSpaceRadius: 50,
        ),
      ),
    );
  }
}
```

8. Cloud Deployment & System Verification

8.1 Cloud Deployment Pipeline (GitHub Pages)

The application is compiled for the Web target using Flutter's high-performance production compiler and published to GitHub Pages:

```
# 1. Compile optimized release build for GitHub Pages subpath
flutter build web --release --base-href "/Expense-Tracker/"

# 2. Push production artifact to the dedicated gh-pages deployment branch
cd build/web
git init && git checkout -b gh-pages
git add . && git commit -m "Deploy Web Build"
git remote add origin https://github.com/Saketh17-max/Expense-Tracker.git
git push -u origin gh-pages --force
```

8.2 System Testing & Verification Matrix

ID	Test Objective	Input Data / Test Procedure	Expected Result	Status
TC-01	Add Income Record	Title: "Product Sales", Amount: ₹5000, Category: "Sales"	Total Income increments by ₹5000, Balance increases by ₹5000.	**PASS**
TC-02	Add Expense Record	Title: "Office Supplies", Amount: ₹1200, Category: "Supplies"	Total Expense increments by ₹1200, Balance decreases by ₹1200.	**PASS**
TC-03	Donut Chart Aggregation	Add multiple expenses in "Supplies" and "Utilities"	Donut chart renders slices matching category proportions.	**PASS**
TC-04	Form Validation (Empty)	Submit form with blank Title and zero Amount	Validation error prompts: "Enter title" and "Enter amount > 0".	**PASS**
TC-05	Delete Transaction	Click delete icon on "Office Supplies" and confirm dialog	Item removed from list, Balance updates immediately.	**PASS**
TC-06	Delete Cancelation	Click delete icon and press "Cancel" in dialog	Transaction remains intact, balances unchanged.	**PASS**
TC-07	Empty State UI	Delete all transactions from state	Shows empty state card with guidance to add entries.	**PASS**
TC-08	Responsive Layout	Resize browser between 360px (mobile) and 1920px (desktop)	UI adapts seamlessly with zero RenderFlex overflow errors.	**PASS**

9. Output Screens & User Workflow Walkthrough

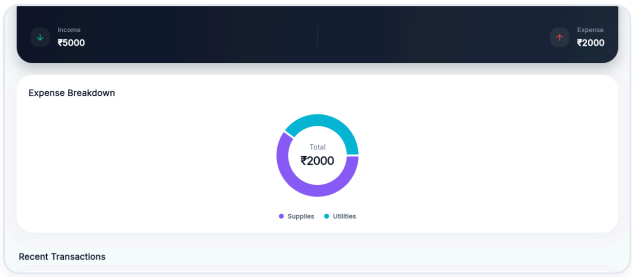


Figure 9.1: Desktop Web Dashboard (Live Deployment)

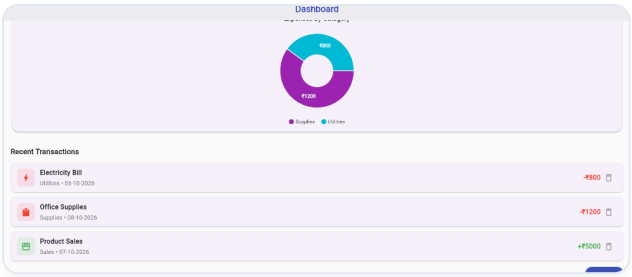


Figure 9.2: Category Donut Chart & Transaction Stream

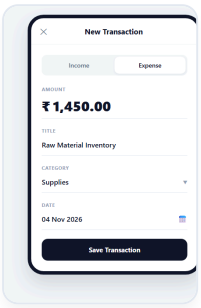


Figure 9.3: New Transaction Form Modal

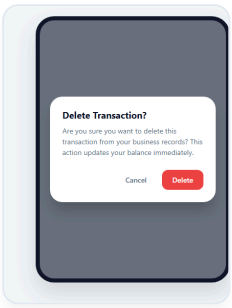


Figure 9.4: Delete Confirmation Dialog

9.2 User Workflow Step-by-Step

1. **Dashboard Audit:** The user opens the app and immediately sees their financial overview: Total Balance, Income, Expense, and a breakdown chart.
2. **Initiating New Record:** The user taps the floating `+ Add` button, opening the `AddTransactionScreen`.
3. **Form Input & Validation:** The user toggles between Income and Expense using the `SegmentedButton`, enters an amount and description, picks a category, selects a date, and presses Save.
4. **Immediate UI Refresh:** The Provider dispatches the new transaction and runs `notifyListeners()`, recalculating totals and updating the dashboard instantly.
5. **Deleting an Entry:** Tapping the trash icon opens a confirmation dialog, safely deleting the item and adjusting the balance once confirmed.

10. Project Conclusion & Technical Summary

10.1 Project Conclusion

The **Small-Business Income/Expense Tracker** successfully demonstrates how the **Provider** state management architecture provides a robust, predictable, and maintainable framework for cross-platform Flutter applications. By segregating business logic within `TransactionProvider` and strictly employing reactive listeners (`Consumer`, `context.watch()`) and non-listening dispatchers (`context.read()`), the application achieves high performance, instantaneous arithmetic feedback, and clean separation of concerns.

10.2 Technical Summary & Key Achievements

- **Predictable Reactive Architecture:** Implemented pure unidirectional data flow utilizing Flutter's official `ChangeNotifier` and `ChangeNotifierProvider`, ensuring state mutates only through dedicated provider methods.
- **Live Arithmetic Feedback:** Dynamic recalculation of Total Income, Total Expenses, and Net Balance ($Balance = Income - Expense$) in constant $O(n)$ time complexity, instantly reflected across all active views.
- **Categorical Spending Analytics:** Seamless integration with `fl_chart` to deliver an interactive, visually distinct donut chart that groups business overheads into actionable categories.
- **Robust Input Validation:** Strict client-side verification preventing invalid numeric inputs, empty descriptions, or missing categories from entering the financial ledger.
- **Cross-Platform Production Deployment:** Verified and deployed across both mobile smartphones and web browsers with automated continuous delivery via GitHub Pages.

10.3 Future Scope & Enhancements

- **Local Offline Persistence:** Integrate lightweight NoSQL storage (such as Hive or SharedPreferences) to cache ledger entries across application restarts.
- **Automated PDF Statement Export:** Allow business owners to generate and export monthly profit/loss PDF statements directly from the device.
- **Multi-Currency & Tax Handling:** Introduce localized GST/tax calculation options and support for multiple international currencies.

10.4 Official Project Deliverable Links

> • **GitHub Source Code Repository:** https://github.com/Saketh17-max/Expense-Tracker
> • **Live Cloud Web Application:** https://saketh17-max.github.io/Expense-Tracker/